Лекция № 02. Организация работы в команде разработчиков. Система контроля версий

Организация работы в команде разработчиков. Система контроля версий: совместимость, установка, настройка

Подготовка специалистов для действий в составе команды является одной из современных задач **подготовки кадров высшей квалификации**. В обычной практике, разработка дипломных и курсовых проектов осуществляется выпускником и студентом единолично, под руководством научного руководителя, при этом не формируются навыки работы в команде, не приобретается опыт работы с инструментальными средствами коллективной разработки проектов, не формируются навыки руководства проектами.

На производстве же молодой специалист сразу получает задание в составе группы. Особенно ярко это проявляется при реализации крупных проектов в области создания программного обеспечения. Отсутствие навыков работы в команде сказывается на производительности молодого сотрудника и его адаптации в рабочем процессе. Важным фактором является целостность ведения всего проекта и контроля над его реализацией. Как часто бывает в реальной работе, увольнение одного из разработчиков приводит к потере целых фрагментов проекта, так как при работе в простой студии, всё находится в компьютере программиста.

Другой не менее важный вопрос - наличие методики реализации проекта, очень важно для любого дела. Привить все эти навыки очень важно со студенческой скамьи. **Современные средства разработки программного обеспечения** частично позволяют решить данные проблемы. Вообще, идея управления циклом создания программного обеспечения командой разработчиков не нова.

Зачастую участники проекта разделены территориально, но даже находясь рядом, крайне неэффективно контактируют между собой, в то время как для успешного проекта необходимо постоянное согласование действий. Можно использовать телефонные переговоры, почту, и другие современные средства коммуникации, но гораздо эффективнее использовать специальный инструментарий, разработанный для этих нужд.

Осознавая эту потребность, компания Microsoft представила новейший пакет инструментальных средств - **Microsoft Visual Studio Team System**. Это не методика, это просто инструмент. Данный пакет тесно интегрирован с другими программными продуктами этой компании, такими как Microsoft Office, Windows SharePoint и др. Необходимость создания единого хранилища и управляющего инструментария для всей информации касаемо разрабатываемого проекта назревала давно. Иначе, в ситуации какого-либо сбоя у одного из участников проекта неизбежны потери информации, ценной для всей команды.

В новом продукте Microsoft роль хранилища и автоматизированного координатора играет **Team Foundation Server**. Сервер визуализирует всю информацию посредством интерактивного web-сайта использующего Windows SharePoint Server. Итак, при **использовании Visual Studio Team System решаются следующие задачи**:

- Повышение предсказуемости успеха проекта.
- Рост производительности труда команды разработчиков за счет понижения сложности процессов проектирования и реализации современных сервисно-ориентированных решений.
- Обеспечение сотрудничества команды путем интеграции коммуникационных и прочих средств, используемых ее членами.
- Расширение возможностей командной работы за счет обеспечения удаленным пользователям доступа к надежному, защищенному и масштабируемому окружению.
- Предоставление команде разработчиков гибкого и расширяемого коммуникационного решения с настраиваемыми сервисами.

Ключевыми членами проектной команды являются: **руководитель проекта, архитектор, разработчик и тестировщик**. Каждую из этих ролей может исполнять как один человек, так и несколько. Второстепенные участники проекта, являющиеся профессионалами в области информационных технологии, также по достоинству оценят использование Team System, так как эта система облегчит им взаимодействие с остальными членами команды.

Научному руководителю курсовых и дипломных проектов, выполняемых в составе команды, необходимо также **решать следующие задачи**:

- Психологическая совместимость.
- Методы подбора групп.
- Сроки введения группового проектирования, первоначальная подготовка.
- Распределение ролей, ротация ролей.

Другой не менее важный вопрос - наличие методики реализации проекта, что очень важно для любого дела..

Основные постулаты и принципы работы в команде

Сегодня основным ресурсом эффективного ведения хозяйственной деятельности предприятия является корпус специалистов. Резко возрастает роль личностей. От их квалификации, деловой активности, умения взаимодействовать друг с другом и достигать результата зависят перспективы развития предприятия. Одним из наиболее востребованных качеств, наряду с профессионализмом, является способность работать в команде. Ключевым фактором эффективной работы команды является способность каждого ее члена "работать на результат". Однако на практике иногда случается так, что психологический аспект смещается с результатов деятельности на межличностную конкуренцию, проявляющуюся в скрытой или явной конфронтации.

В вопросе формирования команды важным элементом является "численность". Пять - подходящее количество людей, двенадцать - слишком много. "7±2" является магическим числом и идеальной численностью команды. В случае если в Вашем коллективе 12 и более человек, рано или поздно естественным образом он поделится на "неформальные команды по интересам", представляя в общем виде разрозненную и неконсолидированную психологически массу. Этот процесс может быть предупрежден руководителем: нужно осознанно объединить сотрудников в команды вокруг определенных профессиональных целей. Когда одна цель будет достигнута, важно вновь и вновь объединять сотрудников в команды, создавая и поддерживая, таким образом, командный дух в коллективе и поддерживая, повышая мотивационный уровень сотрудников.

Состав команды является самой важной предпосылкой высокой производительности компании. При подборе персонала нужно учитывать три фактора.

Фактор первый: профессиональная квалификация. Профессиональные требования зависят от конкретных задач той или иной команды, поэтому до начала формирования команды следует составить список требований, связанных с конкретной задачей.

Фактор второй: личностные характеристики.

Фактор третий: способность работать в команде! Это может быть как предпосылкой, так и результатом развития команды. Без базовой способности сотрудничества с другими членами команды, ориентируясь на результат, ничего не получится. Следовательно, при формировании команды требование к способности работать в ней выдвигается в первую очередь. Необходимо определить хочет ли и может ли кандидат работать в команде, и в какой степени.

Обзор существующих инструментальных средств для командной работы

Наряду с VSTS существуют другие решения для организации командной работы, одним из них является IBM Rational Suite.

Rational Suite - интегрированный набор продуктов, предназначенный для поддержки командной работы над проектом на каждой фазе жизненного цикла разработки информационной системы. Продукт выпущен компанией Rational Software.

IBM Rational Suite является уникальным семейством продуктов, которое позволяет поднять на новый уровень разработку программного обеспечения. Пользователи Rational Suite получают следующие преимущества:

Объединение - Rational Suite объединяет усилия аналитиков, разработчиков и тестировщиков, снимая барьеры, которые обычно существуют между группами сотрудников, работающих над одним проектом.

Оптимизацию - Rational Suite оптимизирует работу сотрудников, предоставляя нужный набор инструментальных средств для выполнения специфических задач каждого члена команды.

Упрощение - Rational Suite упрощает установку, поддержку и совместное использование продуктов, входящих в Rational Suite, снижается их общая стоимость (общая стоимость комплекта значительно ниже стоимости приобретения аналогичных отдельных продуктов).

IBM Rational Suite обеспечивает ускорение разработки на основе функций визуального моделирования, генерации кода и реинжиниринга, обеспечивает поиск и устранение ошибок времени исполнения, проблем с недостатком памяти и производительности.

Программные продукты, составляющие Rational Suite:

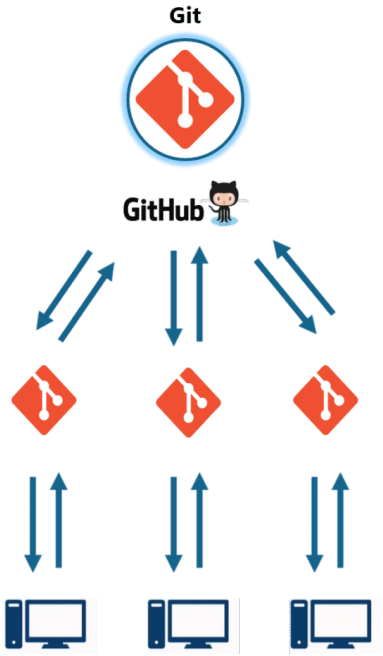
- **IBM Rational PurifyPlus for Windows** - это полный набор автоматизированных инструментальных средств для проведения анализа на этапе выполнения, предназначенный для повышения надежности и производительности приложений, разработанных в среде Windows. Предназначен для приложений, разработанных на Java, Visual C/C++, C#, VB.NET и Visual Basic.
- **IBM Rational Functional Tester for Java and Web** - решение, обеспечивающее расширенное автоматизированное функциональное и регрессионное тестирование Java- и Web -приложений из сред разработки Eclipse IDE, IBM WSAD.
- **IBM Rational Robot** - продукт, автоматизирующий функциональное, регрессионное и конфигурационное тестирование для широкого диапазона типов приложений, включая .NET.
- **IBM Rational Team Unifying Platform** - платформа, предназначенная для обеспечения разработчиков инфраструктурными инструментами, процессами и интеграцией - элементами, необходимыми для более эффективной организации совместной работы. Данное решение объединяет коллектив разработчиков, предоставляя общий доступ к ресурсами разработки, коммуникационным оповещениям и процессам, что необходимо для отслеживания динамики и текущего состояния проекта.
- **IBM Rational Developer Network** - дополнительный ресурс, доступный для всех клиентов Rational, - сеть Rational Developer Network (RDN) , откуда в любой момент времени можно получить учебную информацию, сведения о решениях и советы экспертов.
- **IBM Rational Suite for Technical Developers** - представляет собой полное решение поддержки жизненного цикла ПО, предназначенное для написания очень сложного кода для некоторых наиболее проблемных продуктов и систем, таких, как встроенные приложения и приложения реального времени.
- **IBM Rational Suite Development Studio for UNIX** - представляет собой решение по поддержке всего жизненного цикла ПО, предназначенное для аналитиков, разработчиков и тестировщиков, что позволяет объединить многофункциональные группы разработчиков и поддерживать программные проекты для UNIX, начиная от формулирования требований и до выпуска финального ПО.

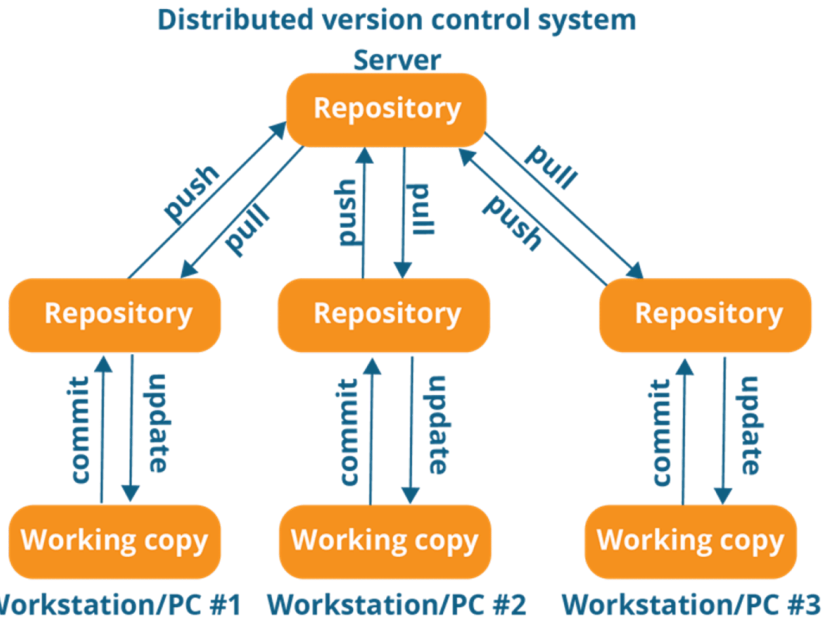
Система контроля версий — программное обеспечение, которое обеспечивает командную работу в рамках одного или нескольких проектов.

Команда разработчиков взаимодействует с консольным или браузерным инструментом для выгрузки кода на сервер, скачивания его на рабочий компьютер и изменения структуры.

Новичкам обычно сложно вникнуть в технические подробности работы системы контроля версий, поэтому мы постараемся объяснить суть простым языком. Не старайтесь сразу погружаться в термины, пользы от этого будет мало.

Представьте, что играете в компьютерную игру. **По мере прохождения миссий происходит сохранение контрольных точек.** Даже если выключить компьютер и зайти в приложение через несколько дней или месяцев, прогресс не потеряется. В любой момент можно выбрать контрольную точку и пройти миссию заново.





Система контроля версий работает по аналогичной схеме. Она хранит все версии проекта и обеспечивает к ним доступ. Любой член команды может взаимодействовать с основной «веткой» проекта или создавать новые.

Самой популярной системой контроля версий является **Git**, поэтому мы не будем подробно останавливаться на альтернативных решениях и расскажем об этом инструменте.

Собрали **основные понятия**, которые будут полезны всем, кто захочет освоить работу в **Git**:

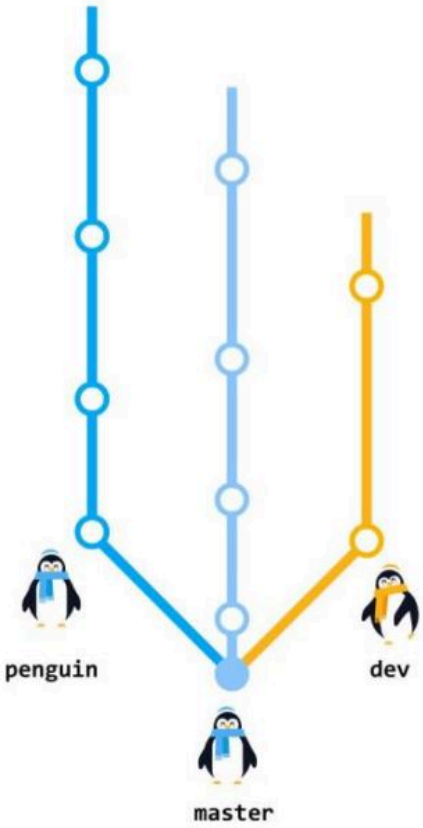
1. **Репозиторий**. Каталог, в котором хранится файловая система проекта. Для каждого проекта создаётся отдельный репозиторий. Существуют локальные и удалённые репозитории. В первом осуществляется работа над проектом на компьютере, а второй выступает в роли хранилища.
2. **Ветка**. Дочерняя версия основного репозитория. Она входит в его состав, но не влияет на работу. После того, как разработчики закончат работу над новой функцией или исправят все баги, можно совместить дочерний и родительский репозитории.
3. **Коммит**. Операция позволяет зафиксировать текущее состояние проекта. После выполнения команды через консоль или использования браузерной версии Git, новая версия добавляется в репозиторий.
4. **Форк**. Копия репозитория, которую можно использовать для изменения исходного кода без отправки изменений в основной репозиторий. Форки часто применяют для open-source проектов, когда любой разработчик может собрать свой проект на основе готового ядра.
5. **Пул и пуш**. Первая операция позволяет выкачивать содержимое репозитория на компьютер, а вторая отправляет измененные файлы на сервер.
6. **Мастер**. Основная ветка репозитория, в которой хранится ядро проекта. В неё добавляют изменения только после тщательного тестирования.
7. **Кодревью**. Процесс проверки кода на соответствие техническому заданию или требованиям внутри команды. Когда один разработчик хочет добавить свой код в ядро, остальные члены команды проверяют его и если проблем нет, происходит обновление главной ветки.

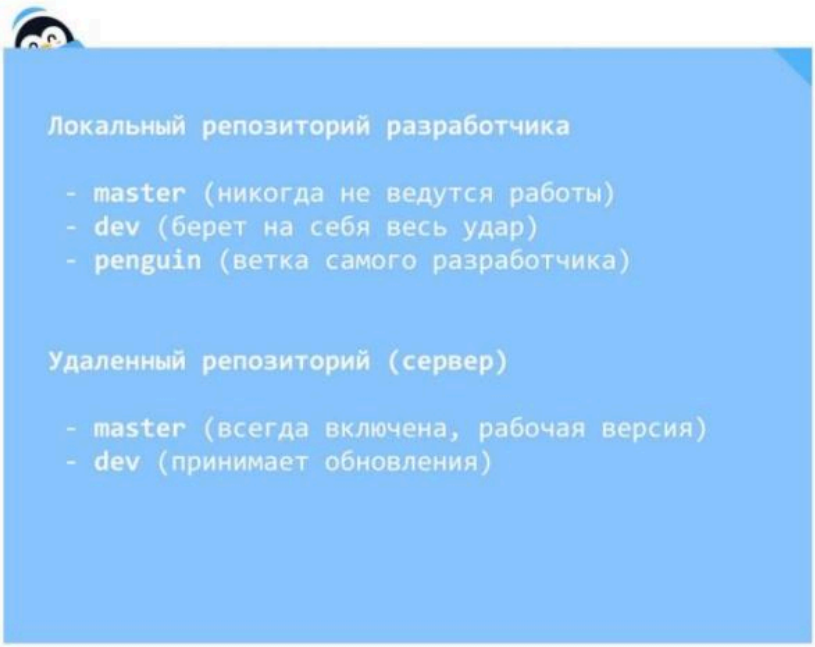
Система контроля версий — полезный инструмент, который нужен не только программистам, работающим в команде. Он пригодится и самостоятельным разработчикам для удобного хранения данных проекта. Если что-то пойдёт не так, в любой момент можно вернуться к контрольной точке и начать заново. .

Какие задачи решает система контроля версий:

1. **Защищает исходный код от потери.** Данные хранятся на удалённом сервере, даже если разработчики удалят файлы с локального компьютера, они останутся в репозитории.
2. **Обеспечивает командную работу**. Программисту не надо использовать инструменты для командной работы и платить за них. Каждый может работать на своём компьютере и обновлять файлы по мере необходимости.
3. **Помогает отменить изменения.** В любой момент можно вернуться к контрольной точке, сравнить исходный код с текущим и обновить главную ветку после ревью.

4. **Распределённая работа.** Необязательно работать с проектом «наживую». Плагин может функционировать на сайте, а программисты будут спокойно создавать новую версию.



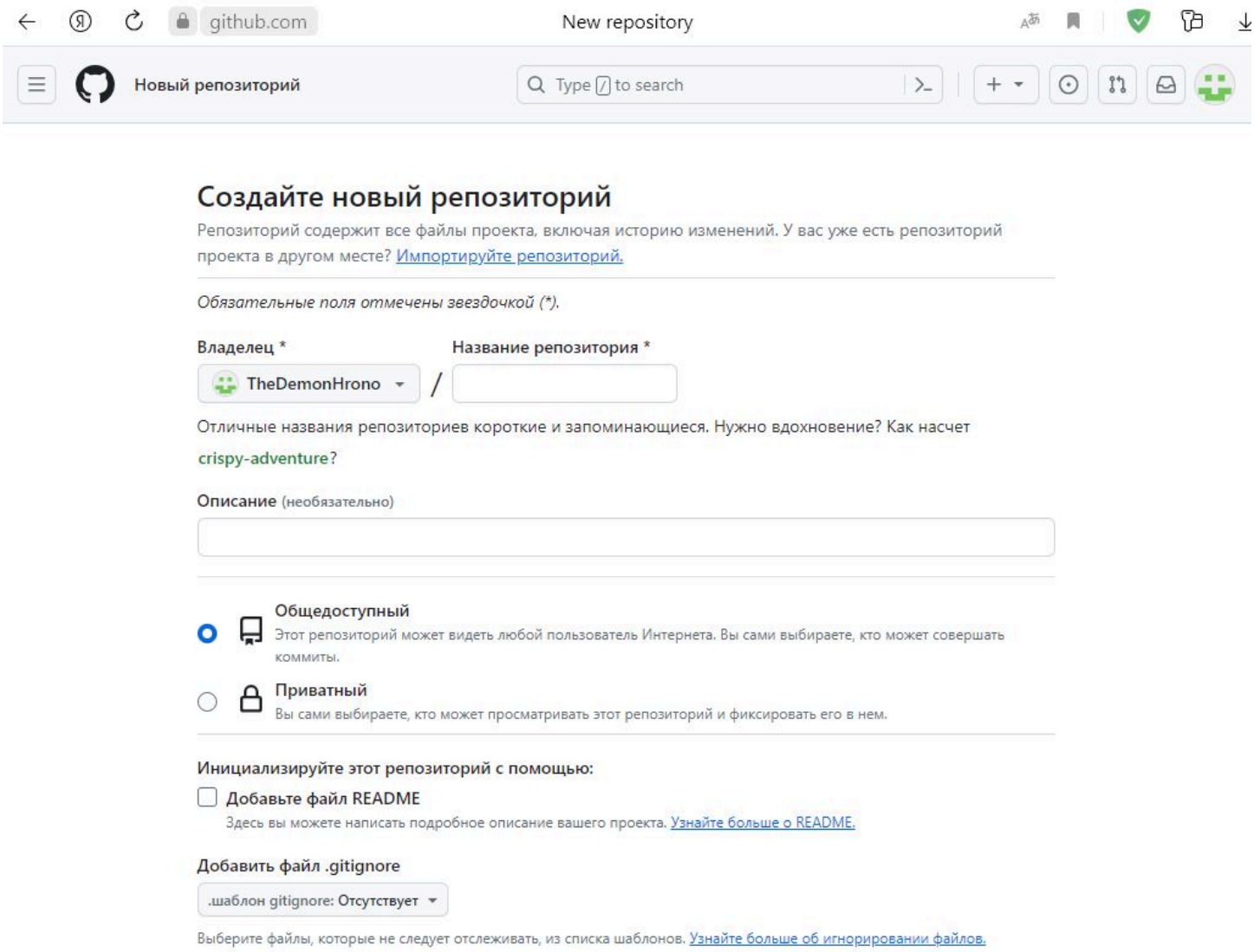


Git или другая система контроля версий также важна в работе программиста, как и сохранение бэкапов. В любой момент могут понадобиться исходники. И если их не будет, появятся дополнительные проблемы.

Как пользоваться системой контроля версий. Уже давно появился стереотип, что программисты очень любят работать с консолью. Новички представляют, что опытные разработчики вводят 20 команд в минуту и не используют веб-версии с user-friendly интерфейсами.

Git в стандартном варианте устанавливается на компьютер и представляет собой командную строку с минималистичным интерфейсом. Разработчики обычно разделяются на два лагеря: **одни пользуются консольной версией, а вторые предпочитают вариант с графической оболочкой**.

Если не хотите осваивать консоль и тратите много времени на ввод простых команд, лучше выбрать **GitHub**. Это крупнейший веб-сервис, на котором размещены тысячи проектов для совместной разработки. **Платформа основана на системе контроля версий Git и обеспечивает удобную командную работу.**



Любой пользователь может зарегистрироваться на сайте GitHub, чтобы создавать репозитории для своих проектов. **Платформа отлично подходит для новичков и даёт полезный опыт.** В системе много проектов с открытым исходным кодом, которые можно дорабатывать по своему усмотрению.

По данным GitHub в системе 56 млн разработчиков, больше 100 млн репозиториев и 3 млн организаций. Клиенты, которые обращаются к программистам за разработкой сайта, сервиса или приложения часто добавляют в список требований использование GitHub.

Git и GitHub не получится освоить за несколько часов, если раньше не было опыта работы с системами контроля версий. Новичкам не надо пытаться сразу выучить все термины. Лучше разобраться с основными понятиями, изучить несколько полезных статей и протестировать Git самостоятельно.

Популярные ошибки при работе с Git

Собрали распространённые ошибки, которые мешают начинающим разработчикам полностью использовать возможности системы контроля версий и создают дополнительные проблемы. **Некоторые ошибки актуальны и для опытных программистов:**

Отказ от использования Git. Многие новички отказываются от системы контроля версий, когда запускают консоль и не понимают, как с ней работать. Можно заменить десктопную версию на GitHub и вообще не взаимодействовать с консолью.

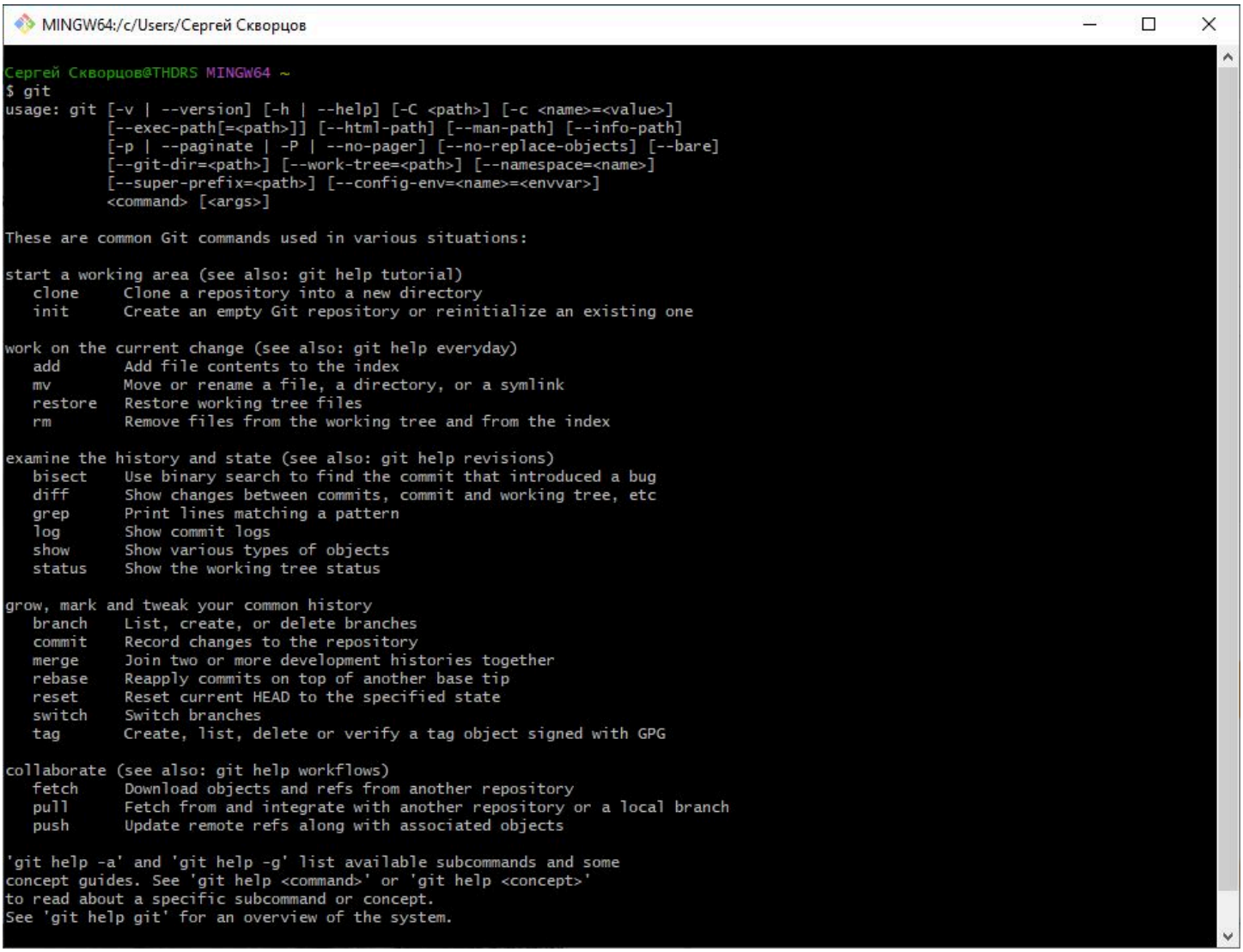
Поверхностное изучение документации. В официальной справке много полезной информации, которая экономит время при работе с репозиториями. Её надо обязательно прочитать и запомнить важные особенности.

Игнорирование правил работы с Git. Идти против системы нельзя, потому что проект окажется под угрозой. Например, не стоит добавлять непроверенные изменения в основную ветку.

Неправильное использование команд. При работе с консольной версией нельзя запускать команду, если до конца не уверены, какую операцию она выполняет.

Ошибочные комментарии к коммитам. После нескольких часов активной работы с кодом лучше не отправлять файлы на сервер, а отложить задачу на потом. Разработчики часто оставляют неправильные комментарии к версиям проекта, а коллеги используют их наработки, чтобы продолжить работу.

Добавление лишних файлов. Если залили файлы, которые не относятся к проекту, их можно удалить или исключить из структуры.



Система контроля версий — must have инструмент для разработчиков с разным опытом. Новичкам лучше сразу освоить его, чтобы использовать для своих проектов и получить дополнительное преимущество перед конкурентами, которые не знают Git или аналоги.

Обеспечение кроссплатформенности информационной системы

> **Кросс-платформенность (межплатформенность)** — способность программного обеспечения работать с несколькими аппаратными платформами или операционными системами. Обеспечивается благодаря использованию высокоуровневых языков программирования, сред разработки и выполнения, поддерживающих условную компиляцию, компоновку и выполнение кода для различных платформ. Типичным примером является программное обеспечение, предназначенное для работы в операционных системах Linux и Windows одновременно.

Кросс-платформенные языки программирования. Кросс-платформенными можно назвать большинство современных высокоуровневых языков программирования. Например, **Си, C++, Free Pascal, FreeBASIC, PureBasic, C#** — **кроссплатформенные языки на уровне компиляции**, то есть для э**тих языков есть компиляторы под различные платформы**. Это позволяет — при надлежащем качестве кода — не переписывать основной движок программы, меняются только особые системозависимые части.

Не менее важны для кросс-платформенности **стандартизованные библиотеки среды выполнения**. В частности, стандартом стала библиотека языка Си (**POSIX**). Из крупных кросс-платформенных библиотек — **Qt, GTK+, FLTK, STL, Boost, OpenGL, SDL, OpenAL, OpenCL**.

Существуют кросс-компиляторы — компиляторы, генерирующие исполняемый код для платформы, отличной от той, на которой запущен сам компилятор.

Кросс-платформенные среды исполнения. PHP, Perl, Python, Tcl и Ruby — кросс-платформенные интерпретируемые языки, их интерпретаторы существуют для многих платформ.

Среды исполнения **ActionScript Virtual Machine, Java Virtual Machine и .NET** также кроссплатформенны, однако на их вход подаётся не исходный текст, а промежуточный код. Поэтому программы, написанные на **ActionScript, Java и C#**, можно запускать под разными операционными системами без предварительной перекомпиляции.

Программные платформы. Программные платформы могут быть либо операционной системой, либо средой программирования , хотя чаще это комбинация того и другого. Заметным исключением из этого правила является Java, которая использует независимую от операционной системы виртуальную машину для своего скомпилированного кода, известного в мире Java как **байт-код**.

Примеры программных платформ:

- BlackBerry
- Android для смартфонов и планшетов (x86, ARM)
- iOS (ARM)
- Microsoft Windows (x86, ARM)
- Интерфейс командной строки, также известный под именами реализации .NET Framework (от Microsoft) и кроссплатформенным вариантом Mono (ранее Novell, а теперь Xamarin)
- Джава
- Веб-браузеры - более или менее совместимы друг с другом, работают с веб-приложениями на JavaScript.
- Linux (x86, PowerPC, ARM и другие архитектуры)
- macOS (x86, PowerPC (на 10.5 и ниже))
- Mendix
- Solaris (SPARC, x86)
- SymbianOS
- SPARC
- PlayStation 4 (x86), PlayStation 3 (на базе PowerPC) и PlayStation Vita (ARM)
- Unix

Кросс-платформенный пользовательский интерфейс

На разных операционных системах и средах — независимо от того, как технически достигнута работа в них — стандартные элементы интерфейса имеют разные размеры. Поэтому простое жёсткое позиционирование элементов интерфейса невозможно — в другой операционной системе (среде) они могут «налезать» друг на друга, «расплываться», «пьянеть». Существует несколько **подходов**:

1. **Единый стиль, общий для всех операционных систем, программы выглядят одинаково под всеми системами. Так работают интерфейсные библиотеки Java наподобие Swing.**
    - Плюс: можно жёстко расставлять элементы управления на манер Delphi, оригинальный стиль.
    - Минус: системе приходится иметь свои экранные шрифты, и стиль отличается от стиля ОС.
2. **Самоадаптирующийся (адаптивный) интерфейс, подстраивающий сетку под реальные размеры элементов управления. Типичные примеры — Qt, wxWidgets, XUL.**
    - Плюс: стандартный стиль операционной системы, очень быстрый и «скинующийся» под Windows XP, Vista и Windows 7, и некоторая автоматизация локализации.
    - Минус: чтобы собрать самоадаптирующуюся (адаптивную) сетку, требуется квалифицированный программист, а также затруднена плотная компоновка.
3. **Гибридный подход реализован в GTK+.**
    - Плюс: шрифты можно брать из системы, а не «тащить» свои, а также некоторая автоматизация локализации.

- Минус: берёт все недостатки от первых двух подходов. Стиль отличается от стиля операционной системы, затруднена плотная компоновка.

В любом случае, в других операционными системами и средах требуется хотя бы минимальное тестирование, так как возможны ошибки компоновки…

Условная компиляция. Даже несмотря на широкую, в общем, стандартизацию аппаратного и программного обеспечения, программисту часто приходится налаживать ветви под разные операционные системы и среды, включая ту или другую посредством условной компиляции. Например, браузер Mozilla Firefox имеет разные комплекты значков под разные операционные системы.

Среды разработки. Ряд IDE, в том числе Free Pascal, Lazarus, Qt Creator, работают на разных операционных системах: Linux, Windows и других.

Эмуляция. Если программа не предназначена для исполнения (запуска) на определённой платформе, но для этой платформы существует эмулятор платформы, базовой для данной программы, то программа может быть исполнена в среде эмулятора.

Обычно исполнение программы в среде эмулятора приводит к снижению производительности по сравнению с аналогичными программами, для которых платформа является базовой, так как значительная часть ресурсов системы расходуется на выполнение функций эмулятора.

Проблемы кроссплатформенной разработки

Есть определенные проблемы, связанные с кроссплатформенной разработкой.

Некоторые из них включают:

- **Тестирование кроссплатформенных приложений может быть значительно более сложным,** поскольку разные платформы могут демонстрировать немного разное поведение или незначительные ошибки. Эта проблема заставила некоторых разработчиков высмеивать кроссплатформенную разработку, называя ее «напиши один раз, отлаживай везде», что соответствует маркетинговому лозунгу Sun Microsystems « напиши один раз, работай где угодно ».
- **Разработчики часто ограничиваются использованием подмножества функций с наименьшим общим знаменателем**, которые доступны на всех платформах. Это может снизить производительность приложения или запретить разработчикам использовать наиболее продвинутые функции каждой платформы.
- **Различные платформы часто имеют разные соглашения о пользовательском интерфейсе**, которые кроссплатформенные приложения не всегда поддерживают. Например, приложения, разработанные для macOS и GNOME, должны располагать наиболее важную кнопку в правой части окна или диалогового окна, тогда как Microsoft Windows и KDE придерживаются противоположного соглашения. Хотя многие из этих различий незначительны, кроссплатформенное приложение, не соответствующее этим соглашениям, может показаться пользователю неуклюжим или чуждым. При быстрой работе такие противоположные соглашения могут даже привести к потере данных , например, в диалоговом окне, подтверждающем, хочет ли пользователь сохранить или отменить изменения в файле.
- **Языки сценариев и виртуальные машины должны переводиться в собственный исполняемый код каждый раз при выполнении приложения,** что снижает производительность. Этот штраф можно уменьшить, используя продвинутые методы, такие как своевременная компиляция; но даже при использовании таких методов некоторые вычислительные затраты могут оказаться неизбежными.
- **Различные платформы** требуют использования собственных форматов пакетов, таких как RPM и MSI. Мультиплатформенные установщики, такие как InstallAnywhere, удовлетворяют эту потребность.

Межплатформенные среды выполнения могут иметь недостатки межплатформенной безопасности, создавая благоприятную среду для межплатформенного вредоносного ПО.

Сервисно - ориентированные архитектуры

> **Сервис-ориентированная архитектура (СОА, англ. service-oriented architecture- SOA)** — модульный подход к разработке программного обеспечения, базирующийся на обеспечении удаленного использования по стандартизированным протоколам распределённых, слабо связанных, легко заменяемых компонентов (сервисов) со стандартизированными интерфейсами.

Программные комплексы, разработанные в соответствии с СОА, обычно **реализуются как набор веб-служб**, взаимодействующих по протоколу SOAP, но существуют и другие реализации (например, на базе jini, CORBA, на основе REST).

Интерфейсы компонентов в сервис-ориентированной архитектуре инкапсулируют детали реализации (операционную систему, платформу, язык программирования) от остальных компонентов, таким образом обеспечивая комбинирование и многократное использование компонентов для построения сложных распределённых программных комплексов, обеспечивая независимость от используемых платформ и инструментов разработки, способствуя масштабируемости и управляемости создаваемых систем.

Получила распространение в конце 1990-х — начале 2000-х годов. С середины 2010-х годов обрела популярность **микросервисная архитектура** — вариант СОА, базирующийся на применении, насколько это возможно, минимальных по размеру сервисов.

Преимуществах SOA:

- **Автономность**. Сервисы, разработанные на основе SOA, не зависят друг от друга, поэтому могут использоваться несколькими приложениями одновременно.
- **Простота в обслуживании**. Чтобы обновить сервисы, разработанные с помощью SOA, не нужно редактировать старую систему полностью. В управлении SOA, всегда есть третья сторона и изменения в ней не будут влиять на вашу систему. Практически в любых ситуациях предыдущий API будет работать просто потому, что он функционирует как раньше .
- **Масштабируемость**. Если перед вами стоит задача обработать большое количество информации, то для SOA это не проблема. Сервис-ориентированную архитектуру легко масштабировать, подключая к ней дополнительные мощности, а это позволяет работать с конфигурацией и подстраивать систему под ваши нужды.
- **Одинаковая структура каталогов**. SOA поддерживает применение «шаблонов», которые помогают выстраивать каталоги под единому принципу.
- **Переиспользование сервисов.** SOA дает возможность повторно применить сервис существующей системы для новых проектов.

Но у SOA есть и недостатки:

- **Сложное управление.** Каждый сервис SOA должен без задержек доставлять сообщения, но их количество может быть больше нескольких миллионов, а это затрудняет управление всеми службами.
- **Большая нагрузка.** В SOA служба проверяет сообщение на соответствие заявленному заранее контракту. В случае, если применяется несколько сервисов, это может привести к увеличению время отклика и снижению общей производительности.
- **Не подходит для GUI**. Приложения с графическим интерфейсом плохо взаимодействуют с SOA, они требуют интенсивный обмен данными, что еще больше нагружает систему.
- **Высокие инвестиционные затраты**. Использование SOA требует большого количества инвестиций: организационных, экономических, технических и психологических.

Отличия SOA от MSA (микросервисной архитектуры)

Отличия заключаются в **технических особенностях и реализации этих подходов**. Так, например, использование сервис-ориентированной архитектуры невозможно без ESB, централизации и наличия крупных сервисов. Система, спроектированная с помощью SOA, многослойна и по принципу работы напоминает монолитную архитектуру. Но сервис-ориентированная архитектура намного сложнее монолитной за счет глубоких связей и относительной децентрализованности.

В свою очередь, MSA — это подход, при котором система разрабатывается на **основе небольших самостоятельных серверов**. В отличие от SOA, **микросервисы избегают повторное использование и предпочитают дублирование.** Повторное использование серверов предполагает связанность между ними,

а микросервисная архитектура — это максимальная децентрализованность и возможность распределить нагрузку между компонентами. Другими словами, **микросервисная архитектура помогает разбить сервисы по бизнес-отраслям**. Каждый сервис имеет все необходимые для самостоятельного функционирования элементы и существует как независимый процесс. Использовать MSA лучше в проектах с большим коллективом разработчиков, объемной и сложной архитектурой, продуктами, с резко меняющимся трафиком, приложениями, которые требуют частых обновлений.